CONTRIBUTING GUIDE

Brusselator Reaction-Diffusion Solver

Developer documentation for setting up the project, understanding its structure, making changes, building the code and running tests.

Who should use this guide?
Developers and students who want to understand the codebase, investigate the numerical methods, fix issues, add functionality or submit a contribution.

This guide describes the repository layout and commands currently documented for the project. Where a convention has not been explicitly established, it is labelled as a recommended practice.

1. Purpose and contribution scope

This guide explains how to become productive as a contributor to the Brusselator reaction-diffusion solver. It covers the development environment, code organisation, build process, tests, numerical validation, parameter studies and a suggested contribution workflow.

The project is a C++ implementation of a two-dimensional, time-dependent Brusselator reaction-diffusion system. It uses finite differences for spatial discretisation and provides three time-integration methods:

- **Explicit Euler**.
- **Classical fourth-order Runge-Kutta (RK4)**.
- **Implicit Backward Euler**, with Newton iterations for the nonlinear solve.

Eigen is used for matrix operations and sparse linear algebra. Results are written in VTK format for post-processing and visualisation. Python and Bash support component testing, validation and parameter studies.

Typical contributions include numerical bug fixes, improvements to the discretisation or time integrators, additional unit/component tests, improvements to parameter studies, and clearer documentation.

2. Codebase map

The repository is organised by responsibility. The following overview reflects the current project layout:

```
scientific_computing_project/
├── CMakeLists.txt
├── README.md
├── requirements.txt
├── startContainer.sh
├── build/                          # Generated build directory
├── configs/                        # Docker and development configuration
│   ├── Dockerfile
│   ├── custom_bashrc
│   ├── jupyter_lab_config.py
│   ├── neovim_config
│   └── python.md
├── include/                        # C++ declarations / headers
│   ├── brusselator.hpp
│   ├── integrator.hpp
│   ├── output.hpp
│   └── simconfig.hpp
├── src/                            # C++ implementations
│   ├── brusselator.cpp
│   ├── integrator.cpp
│   ├── output.cpp
│   └── solver.cpp
├── resources/                      # Reference data
│   └── final_output_30s.vts
├── scripts/                        # Parameter variation scripts
│   ├── varyCb.sh
│   └── varyDt.sh
├── tests/
│   ├── unit/                       # C++ unit tests
│   │   ├── test_jacobian.cpp
│   │   ├── test_laplacian.cpp
│   │   └── test_reaction.cpp
│   └── component/                  # Python component tests / notebooks
│       ├── parameter_cb_rk4.ipynb
│       ├── parameter_dt_implicit.ipynb
│       └── test_component.py
├── validation/
│   └── validation.ipynb
└── results/                        # Generated simulation output
```

Path	Responsibility / contributor entry point
include/	Header declarations for the solver, integrators, output and simulation configuration.

Path	Responsibility / contributor entry point
src/	C++ implementation files, including the solver entry point.
CMakeLists.txt	Project build configuration and executable/test targets.
tests/unit/	Focused C++ tests for numerical building blocks.
tests/component/	Python component test and notebooks for parameter studies.
validation/	Notebook-based comparison against the supplied reference solution.
scripts/	Shell scripts that run repeated simulations and collect their output.
resources/	Reference VTK data used in validation.
configs/	Docker image and development-environment configuration.
results/ and build/	Generated output/build artifacts; not source-code locations.

Orientation tip: Before changing a numerical routine, trace the relevant declaration in include/, its implementation in src/, the call path from solver.cpp, and the tests that exercise the behaviour.

3. Development environment

3.1 Requirements

- Docker Engine or Docker Desktop.
- A Bash-capable terminal.
- The complete repository, including configs/Dockerfile and CMakeLists.txt.
- For graphical ParaView use from a Windows host, the WSL2/WSLg display setup expected by startContainer.sh.

The documented workflow builds and runs the project inside Docker. This avoids requiring contributors to install the C++ toolchain and project libraries directly on the host.

3.2 Build the development image

Run this command from the repository root on the host computer (the directory containing CMakeLists.txt and configs/):

```
docker build -t brusselator-image -f configs/Dockerfile .
```

The final dot is the Docker build context. The Dockerfile is located in configs/, so the -f option points Docker to that file.

3.3 Start the container

```
chmod +x startContainer.sh
./startContainer.sh
```

Run the script from the repository root. It mounts the current project directory into the container at /workspace and configures display-related environment variables and mounts for graphical applications.

```
hostname
cd /workspace
pwd
ls
```

The configured hostname is mycfdsolver-container. The project root inside the container is /workspace. Because the repository is bind-mounted, source changes and generated results are visible from the host.

Environment boundary: Commands in the remaining sections are run inside the container unless they are explicitly identified as host commands.

4. Build the project

From inside the container, configure and compile from the build directory:

```
cd /workspace
mkdir -p build
cd build
cmake ..
make -j$(nproc)
```

CMake reads the root CMakeLists.txt and generates the build system. GNU Make performs the compilation. The parallel option uses the available CPU cores. The configured build produces the main **brusselator** executable and the **unit_tests** executable.

```
ls -l brusselator unit_tests
```

After modifying C++ source or header files, rebuild from /workspace/build using:

```
cmake ..
make -j$(nproc)
```

If CMake configuration or compilation fails, resolve the first reported error before interpreting later errors, which may be consequences of the initial failure.

5. Running the solver while developing

5.1 Command-line interface

```
./brusselator <solver> <nx> <ny> <t_end> <dt> <write_interval> <Ca> <Cb>
```

Argument	Meaning
solver	explicit, implicit or rk4
nx, ny	Grid dimensions in the x- and y-directions
t_end	Final simulation time
dt	Time-step size
write_interval	Interval for writing output
Ca, Cb	Brusselator reaction parameters

Examples (run from /workspace/build):

```
./brusselator explicit 40 40 30 0.01 1 1 3
./brusselator rk4 40 40 30 0.01 1 1 3
./brusselator implicit 40 40 30 0.01 1 1 3
```

The solver writes VTK .vti output files into method-specific result directories. The implicit solver also uses Newton iterations; inspect its output and the relevant history file when investigating nonlinear convergence behaviour.

6. Testing

6.1 C++ unit tests

Unit tests are located in tests/unit/:

- **test_laplacian.cpp** — tests the Laplacian operator.
- **test_reaction.cpp** — tests the reaction terms.
- **test_jacobian.cpp** — tests the Jacobian used by the implicit solver.

The unit-test executable is built by the same CMake/Make process as the solver. From the container:

```
cd /workspace/build
cmake ..
```

```
make -j$(nproc)
./unit_tests
```

When changing a numerical building block, add or update a focused unit test that exercises the changed behaviour, including relevant edge cases where appropriate.

6.2 Python component test

The Python component test is tests/component/test_component.py. It checks the solver at a higher level by invoking it as a component.

Create and activate the virtual environment from the project root:

```
cd /workspace
python3 -m venv .venv
source .venv/bin/activate
```

Install the dependencies listed by the project:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Run the component test:

```
pytest tests/component/test_component.py
```

The compiled solver must be available in /workspace/build. If pytest is not available, check that .venv is active and that requirements.txt was installed.

6.3 Notebook-based tests and validation

The component notebooks are:

- **tests/component/parameter_cb_rk4.ipynb** — Cb variation with RK4.
- **tests/component/parameter_dt_implicit.ipynb** — time-step variation with the implicit solver.

The validation notebook is validation/validation.ipynb. It compares computed output with resources/final_output_30s.vts.

Activate the Python environment and launch the relevant notebook from its directory:

```
cd /workspace
source .venv/bin/activate
cd /workspace/validation
jupyter notebook
```

For the component notebooks, use:

```
cd /workspace
source .venv/bin/activate
cd /workspace/tests/component
jupyter notebook
```

Open the URL printed in the terminal in your host browser. With the supplied container port mapping, the notebook interface is expected at:

```
http://127.0.0.1:8888/tree
```

Stop the notebook server with Ctrl+C in the terminal where it is running.

7. Parameter-study scripts

The Bash scripts in scripts/ automate repeated simulations. Execute them from /workspace/scripts because their relative paths point to ../build and ../results.

7.1 Cb variation using RK4

```
cd /workspace/scripts
chmod +x varyCb.sh varyDt.sh
./varyCb.sh Variation_Cb Cb 1.5 2.0 2.5 3.0
```

The script uses RK4, a 40 × 40 grid, end time 30, dt = 0.01, write interval 1 and Ca = 1; it changes Cb for each run. It creates a directory for each supplied Cb value and moves the generated script and VTK output into that directory.

7.2 Time-step variation using Implicit Backward Euler

```
./varyDt.sh Variation_dt dt 0.01 0.02 0.03 0.04 0.05
```

The script uses the implicit solver, a 40 × 40 grid, end time 5.0, write interval 1, Ca = 1 and Cb = 3; it changes dt for each run. It moves VTK output, start.sh and newton_history.txt into the corresponding parameter-specific directory.

The solver creates method-specific directories such as Runge-Kutta/ and Implicit_Euler/. The scripts create additional directories for the parameter study and move output files into them. Both directory levels are expected.

The scripts use their own predefined settings for parameters that are not varied. Review the script before changing its defaults or using it for a different study.

8. Numerical and code-change considerations

The following are recommended contributor practices. They are guidance for making changes safely; they are not presented as pre-existing mandatory project conventions.

8.1 Keep numerical changes traceable

- Identify the governing term or numerical operation being changed (diffusion, reaction, time integration, Jacobian construction or output).
- Keep changes local to the relevant implementation where practical.
- Check that declarations in include/ remain consistent with definitions in src/.
- For implicit-method changes, consider both the nonlinear residual/Jacobian behaviour and the Newton iteration outcome.
- For time-integration changes, consider stability and temporal accuracy separately; a stable run is not by itself evidence of an accurate solution.

8.2 Add evidence with a numerical change

- Add or update a unit test for the smallest affected numerical component.
- Run the component test if the solver's end-to-end behaviour may be affected.
- Run the validation notebook when the change can alter the computed solution.
- For parameter-sensitive changes, run an appropriate parameter study and retain the relevant output or plots for review.
- State the grid, time step, final time and physical/reaction parameters used when reporting results.

8.3 Output and generated files

The build/ and results/ directories contain generated artifacts. Do not treat generated output as source code. Avoid committing large simulation outputs unless they are deliberately required as reference data or are specifically requested for a contribution. The existing reference file in resources/ is part of the documented validation workflow.

9. Recommended contribution workflow

1. **Understand the issue.** Reproduce the behaviour before editing. Record the solver method, grid, time step, end time and reaction parameters.
2. **Locate the relevant code.** Trace the relevant header in include/, implementation in src/, and existing tests in tests/.
3. **Make a focused change.** Keep the change limited to the intended behaviour and avoid unrelated formatting or generated-file changes.
4. **Build.** Run cmake .. and make -j$(nproc) from /workspace/build.

5. **Test.** Run ./unit_tests and, when relevant, pytest tests/component/test_component.py.

6. **Validate numerical behaviour.** Use validation/validation.ipynb or the relevant parameter-study notebook/script when the change affects numerical results.

7. **Review the diff.** Check that only intended source, test and documentation files are included.

8. **Explain the contribution.** Describe the problem, the approach, the tests performed and any known limitations in the pull request.

10. Pull-request checklist

- The change addresses one clearly described problem or improvement.
- The project builds successfully using the documented CMake and Make commands.
- Relevant unit tests pass.
- Relevant component tests pass, if applicable.
- Numerical validation or parameter-study evidence is included when the solution may change.
- No accidental build artifacts, temporary files or unrelated result files are included.
- Documentation is updated if commands, arguments, file locations or behaviour have changed.
- The pull-request description states what changed, why it changed, how it was tested and any limitations.

11. Useful commands

Purpose	Command
Build Docker image (host, repository root)	docker build -t brusselator-image -f configs/Dockerfile .
Start container (host, repository root)	chmod +x startContainer.sh && ./startContainer.sh
Build solver and tests (container)	cd /workspace/build && cmake .. && make -j$(nproc)
Run unit tests	cd /workspace/build && ./unit_tests
Activate Python environment	cd /workspace && source .venv/bin/activate
Install Python dependencies	pip install -r requirements.txt
Run component test	pytest tests/component/test_component.py
Run Cb parameter study	cd /workspace/scripts && ./varyCb.sh Variation_Cb Cb 1.5 2.0 2.5 3.0
Run dt parameter study	cd /workspace/scripts && ./varyDt.sh Variation_dt dt 0.01 0.02 0.03 0.04 0.05
Open validation notebook	cd /workspace/validation && jupyter notebook
Open component notebooks	cd /workspace/tests/component && jupyter notebook
Launch ParaView	paraview

12. Project-specific details to confirm before contributing

Some repository policies and implementation details are not specified in the available project documentation. Before making a contribution that depends on them, check the current repository files or ask the project maintainer:

- The exact compiler version and supported host operating systems.
- The precise CMake options and how the unit-test framework is configured.
- Any project-specific C++ formatting, naming or branching conventions.

- The accepted numerical tolerances and formal validation criteria.
- The prescribed admissible range for dt in the implicit solver.
- Whether contributors should attach generated plots, datasets or other artifacts to pull requests.

Do not assume a numerical result is correct solely because the executable completes. For numerical changes, explain the validation method and provide enough run parameters for another developer to reproduce the result.

13. Final note

A useful contribution is not only a code change that builds. It is a change that another developer can understand, reproduce and review. Keep the implementation focused, test the affected behaviour, and document the numerical evidence behind changes to the solver.